

Seminário XSStrike

 por Matheus Barros

Cross-site Scripting (XSS)

OWASP

Categorizado como A03:2021 no OWASP Top 10 (juntamente com Injection), destacando que a injeção de scripts maliciosos no lado do cliente é uma falha crítica e prevalente.

Exemplo

```
<script>alert('XSS PoC')</script>
```

A aplicação confia cegamente nos dados fornecidos pelo usuário e os renderiza diretamente no navegador, que executa o script como se fosse parte legítima da página.

Vulnerabilidade

É uma falha que permite a um atacante injetar scripts maliciosos (geralmente JavaScript) em páginas web que são subsequentemente vistas por outros usuários, fazendo com que o navegador da vítima execute o script.

Consequências

As ramificações incluem o roubo de sessões e cookies de usuários, captura de credenciais (keylogging), redirecionamento para sites maliciosos e a completa manipulação do conteúdo exibido para a vítima.

Exemplo de Código Vulnerável

```
// Rota que renderiza uma página de resultados de busca
app.get('/search', (req, res) => {
  const searchTerm = req.query.q;
  // VULNERÁVEL: O termo da busca é inserido diretamente no HTML!
  // Motores de template como o EJS usam <%- ... %> para output não escapado.
  res.render('search-results', {
    results: `Você buscou por: <%- searchTerm %>`
  });
});
```

O Que é XSStrike?



Scanner Avançado

O XSStrike é um scanner de XSS e gerador de payloads. Ele identifica e explora vulnerabilidades, empregando técnicas de fuzzing e análise de resposta HTTP para alta precisão.

Principais Recursos do XSSStrike



Crawling

Mapeia aplicações web de forma recursiva para encontrar superfícies de ataque.



Geração de Payloads Inteligente

Cria payloads contextualmente relevantes para maximizar a eficácia da exploração.



Bypass de WAFs

Contorna proteções como ModSecurity e Cloudflare, crucial para testes em ambientes reais.



Deteção de Reflected/Stored XSS

Abrange os tipos clássicos de XSS.



Varredura de DOM XSS

Identifica vulnerabilidades XSS do lado do cliente.



Análise Semântica

Compreende o comportamento da aplicação para refinar os payloads.

Pirâmide de Testes



Como o XSSStrike Funciona

1

Coleta de Parâmetros

Identifica pontos de entrada vulneráveis em requisições GET e POST.

2

Injeção de Fuzzing

Testa milhares de payloads XSS, buscando respostas que indiquem vulnerabilidade.

3

Análise de Resposta

Interpreta o código-fonte da resposta HTTP para confirmar a execução do payload.

4

Análise Contextual e Encode

Adapta e codifica payloads com base no contexto (HTML, JS, URL) para evasão de filtros.

5

Técnicas de Bypass

Emprega ``html entities``, ``javascript unicode escapes`` e ``hex encoding`` para contornar proteções.

Comandos Essenciais

Exemplos de comandos práticos para operar o SQLMap em cenários comuns de teste de penetração:

```
xsstrike -u "URL"
```

Realiza um scan focado em uma URL para encontrar XSS Refletido.

```
xsstrike -u "URL" --crawl
```

Mapeia todo o site a partir de uma URL inicial em busca de alvos.

```
xsstrike -r "arquivo.req"
```

Testa uma requisição complexa (POST, com cookies) salva a partir do Burp Suite.

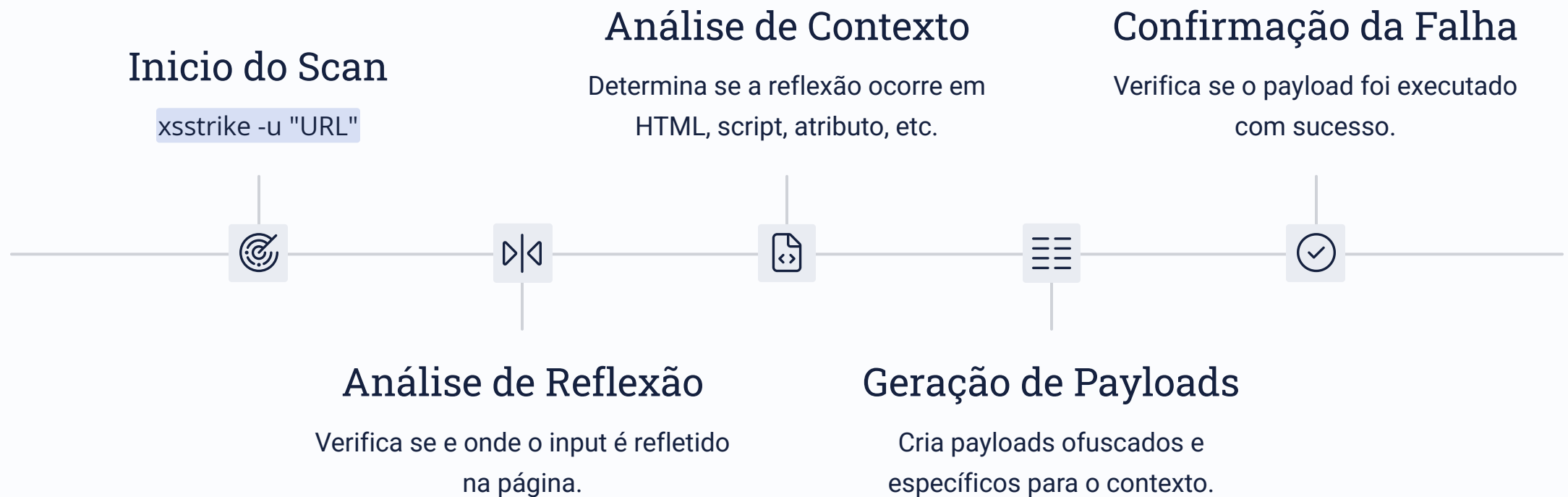
```
xsstrike -u "URL" --fuzzer
```

Utiliza um motor de fuzzing para testes mais agressivos e profundos.

```
xsstrike -u "URL" --blind
```

Testa payloads de Blind XSS, que enviam um callback para um serviço externo.

Fluxo de Ataque



Integrações

Ferramenta	Integração
GitHub Action	Possível utilizar actions da comunidade
JUnit	Leitura de resultados JSON
BurpSuite	Exporta requests que podem ser usados como parâmetro

Vantagens x Desvantagens

✓ **Inteligência Contextual:** Gera payloads mais eficazes e com menos falsos positivos que fuzzers brutos.

✓ **Recursos Avançados:** Detecção nativa de WAFs e scanner de vulnerabilidades DOM XSS.

✓ **Velocidade:** Mais rápido que alternativas que dependem de um navegador headless (como o Selenium).

✗ **Aplicações Modernas (SPAs):** Pode ter dificuldade em "ver" reflexões em SPAs, exigindo o fluxo de trabalho com Burp Suite.

✗ **Potencialmente Ruidoso:** Um scan com --fuzzer ou --crawl em um ambiente monitorado pode gerar muitos alertas.

✗ **CLI:** A ausência de uma interface gráfica pode ser um desafio para iniciantes.

Legalidade e Limitações

Permissão Explícita

O uso do XSSStrike é estritamente limitado a sistemas para os quais você possui permissão expressa e por escrito do proprietário. Ações não autorizadas são ilegais e passíveis de penalidades.

Implicações Legais no Brasil

No Brasil, a Lei nº 12.737/2012 (Lei Carolina Dieckmann) tipifica crimes informáticos, prevendo graves consequências jurídicas para invasões e acessos indevidos.

Nem Todo XSS é Explorável

O XSSStrike não é uma solução mágica; nem todas as vulnerabilidades de XSS são automaticamente exploráveis, especialmente com filtros de segurança robustos e uma Content Security Policy (CSP) bem configurada.

Conhecimento Essencial

Apesar de sua automação, o uso efetivo do XSSStrike requer conhecimento sólido de JavaScript, HTML e de como os navegadores interpretam o código para analisar os resultados e criar exploits manuais.

Casos Reais

Caso	Vulnerabilidade	Impacto
Samy Worm - MySpace (2005)	Stored XSS no perfil do usuário que se auto-replicava.	+1M de "amigos" para o criador em 20 horas, causando a queda do site.
Twitter (2009)	XSS com o evento "onMouseOver" que criava um worm.	Retweets e pop-ups automáticos ao passar o mouse sobre um link,
eBay - UK (2014)	XSS persistente em descrições de produtos	Redirecionava compradores a um site falso que roubava login
British Airways (2018)	Injeção de JavaScript (Magecart) – XSS em componente de bagagem	Roubo de dados de ≈380 000 transações

Ferramentas Similares

BeEF

Foca na **pós-exploração** de XSS

Burp Suite

Ferramenta DAST padrão para XSS e outras falhas.

OWASP ZAP

Alternativa open-source ao Burp Suite

Quando Usar x Evitar

✓ **Teste de segurança:** DAST em APIs e sites antes de ir para produção

✓ **Ethical Hacking:** durante *bug bounty*

✓ **Aprendizado:** laboratórios de vulnerabilidades para estudo prático

✗ **Aplicações críticas em produção:** Sem um ambiente de homologação (staging), o crawler ou o fuzzer podem causar alta carga ou DoS.

✗ **Unethical Hacking:** uso para fins criminosos

✗ **Exploração não autorizada:** análise de aplicações sem permissão do proprietário